Évaluation – CPP04

Tests préliminaires

Si une triche est suspectée, l'évaluation s'arrête ici. Utilisez le flag « Cheat » pour le signaler. Prenez cette décision avec prudence.

Prérequis

Le code doit compiler avec **c++** et les flags **-Wall -Wextra -Werror**. Le projet doit respecter le standard **C++98**. Les fonctionnalités et conteneurs de **C++11** (et versions ultérieures) ne sont donc **PAS** autorisés.

Chacun des points suivants signifie que vous ne devez pas noter l'exercice :
• Une fonction est implémentée dans un header (sauf les fonctions template).
• Le Makefile ne compile pas avec les flags requis et/ou utilise un compilateur autre que **c++**.

Chacun des points suivants signifie que vous devez attribuer le flag « Forbidden function » :
• Utilisation d'une fonction du langage C (par exemple **printf**, **fprintf**, etc.).
• Utilisation d'une fonction interdite par les consignes de l'exercice.
• Utilisation de **using namespace** ou du mot-clé **friend**.
• Utilisation d'une bibliothèque externe ou de fonctionnalités provenant d'une version autre que C++98.

✓ Oui ✕ Non

Exercice 00 : Polymorphism

Comme d'habitude, les tests doivent être assez nombreux pour prouver que le programme fonctionne comme prévu. Sinon, ne notez pas cet exercice. Une classe d'interface sans destructeur virtuel ne respecte pas la forme canonique orthodoxe : ne notez pas cet exercice.

Vérification initiale

Il existe une classe **Animal** qui possède un attribut (une chaîne de caractères appelée **type**). Vous devez pouvoir instancier cette classe et l'utiliser.

✓ Oui ✕ Non

Inheritance

Il existe au moins deux classes qui héritent de **Animal** : **Cat** et **Dog**. Les affichages des constructeurs et des destructeurs doivent être clairs. Demandez à l'étudiant de les expliquer.

✓ Oui ✕ Non

Easy derived class

L'attribut **type** doit être initialisé avec la valeur adaptée à chaque animal. **Cat** doit avoir « Cat » et **Dog** doit avoir « Dog ».

✓ Oui ✕ Non

Animal

En appelant **makeSound()**, vérifiez que la fonction appropriée est appelée. **makeSound()** doit être **virtual** : vérifiez-le dans le code.
virtual void makeSound() const;
La valeur de retour n'a pas d'importance, mais le mot-clé **virtual** est obligatoire.

Il doit y avoir un exemple avec **WrongAnimal** et **WrongCat** qui n'utilisent pas le mot-clé **virtual** (voir le sujet). Le **WrongCat** doit afficher le son de **WrongCat** uniquement lorsqu'il est utilisé comme un **WrongCat**.

✓ Oui ✕ Non

Exercice 01 : I do not want to set the world on fire

Comme d'habitude, les tests doivent être assez nombreux pour prouver que le programme fonctionne comme prévu. Sinon, ne notez pas cet exercice. Une classe d'interface sans destructeur virtuel ne respecte pas la forme canonique orthodoxe : ne notez pas cet exercice.

Concrete Animal

Il existe une nouvelle classe appelée **Brain**. **Cat** et **Dog** possèdent l'attribut privé requis de type **Brain**. Cet attribut ne doit pas se trouver dans la classe **Animal**. La classe **Brain** affiche des messages spécifiques lors de sa création et de sa destruction.

✓ Oui ✕ Non

Concrete Brain

La copie d'un **Cat** ou d'un **Dog** doit être une **deep copy**. Testez, par exemple, le cas suivant :

```
Dog basic;
{
Dog tmp = basic;
}
```

✓ Oui ✕ Non

Si la copie est une **shallow copy**, **tmp** et **basic** utiliseront le même **Brain**, qui risque d'être supprimé deux fois. La copie doit créer un nouveau **Brain** et copier son contenu.

✓ Oui ✕ Non

Destruction chaining

Les destructeurs de **Animal** doivent être **virtual**. Demandez à l'étudiant d'expliquer pourquoi le destructeur de **Animal** doit être virtuel. Testez-le.

✓ Oui	✕ Non

Assignment and copy

Les opérateurs d'affectation et les constructeurs de copie de **Cat** et **Dog** doivent respecter les mêmes exigences. Une **deep copy** signifie qu'il faut créer un nouveau **Brain**. Vérifiez que la forme canonique orthodoxe est implémentée, sans constructeur de copie ni opérateur d'affectation manquant. Rien ne doit être **public** sans raison. Le code doit être propre.

✓ Oui	✕ Non

Exercice 02 : Abstract class

Comme d'habitude, les tests doivent être assez nombreux pour prouver que le programme fonctionne comme prévu. Sinon, ne notez pas cet exercice.

Abstract class

Il existe une classe **Animal** identique à celle du sujet. La fonction **Animal::makeSound()** est une fonction **pure virtual**. Elle doit ressembler à : **virtual void makeSound() const = 0;**
La partie « **= 0** » est obligatoire. Vous ne devez pas pouvoir instancier **Animal**. Si vous pouvez le faire, l'exercice échoue.

✓ Oui	✕ Non

Concrete Animal

Les classes **Cat** et **Dog** sont toujours présentes et fonctionnent exactement comme dans l'exercice 01.

✓ Oui	✕ Non

Exercice 03 : Interface and recap

Comme d'habitude, les tests doivent être assez nombreux pour prouver que le programme fonctionne comme prévu. Sinon, ne notez pas cet exercice.

Interfaces

Les interfaces **ICharacter** et **IMateriaSource** sont présentes exactement comme demandé dans le sujet.

✓ Oui	✕ Non

MateriaSource

La classe **MateriaSource** est présente et implémente **IMateriaSource**. Ses méthodes fonctionnent comme prévu.

✓ Oui	✕ Non

Concrete Materia

Les classes concrètes **Ice** et **Cure** héritent de **AMateria**. Leurs méthodes sont correctement implémentées et leurs affichages sont corrects. **AMateria** est abstraite : **clone()** est une fonction **pure virtual**. **AMateria** possède un attribut **protected** pour stocker le type.

✓ Oui	✕ Non

Character

La classe **Character** est présente et implémente **ICharacter**. Elle peut porter au maximum 4 **Materia**. Les fonctions membres sont implémentées comme demandé. Le constructeur de copie et l'opérateur d'affectation sont implémentés comme demandé (**deep copy**).

✓ Oui	✕ Non

Ratings

N'oubliez pas de cocher le flag correspondant à votre évaluation.

✓ OK				
E : Travail vide	I : Travail incomplet	W : Compilation invalide	✕ Cheat	■ Crash
▲ Situation préoccupante	E : Fuites	F : Fonction interdite		